AI prompt cell

Show code

google/gemini-2.5-flash

- Interpretation sections for each experimental phase

Please evaluate:
1. Whether the modeling flow is logically structured.
2. Whether feature selection was applied correctly

Submit Text

1. **Explicit Threat Model:** State the specific insider threat scenarios (e.g., data exfiltration, sabotage, espionage) the solution aims to detect, as different threats manifest differently.
2. **Handling Imbalanced Data:** Detail the strategies used to address the severe class imbalance in CERT r4.2 (e.g., oversampling, undersampling, SMOTE, cost-sensitive learning, class weights) and their impact.
3. **Hyperparameter Tuning & Cross-Validation:** Thoroughly document the hyperparameter tuning process for *all* models (e.g., GridSearchCV, RandomizedSearchCV, Bayesian Optimization) and the cross-validation strategy employed (e.g., K-fold, stratified K-fold) for robust evaluation.
4. **Ablation Studies:** Conduct experiments to quantify the contribution of specific components, such as the value added by domain-driven latent features or the impact of feature reduction.
5. **Error Analysis:** Go beyond aggregate metrics by performing qualitative error analysis. Examine specific false positives and false negatives to understand model weaknesses and potential biases.
6. **Statistical Significance Testing:** Use appropriate statistical tests (e.g., McNemar's test for comparing classifiers, paired t-tests) to determine if performance differences between models are statistically significant, rather than just observed.
7. **Interpretability of MLP:** While MLP is a black-box, consider techniques like SHAP or LIME to provide *some* local interpretability for critical predictions, especially in a security context.
8. **Computational Cost/Scalability:** Briefly discuss the computational resources required for training and inference, especially relevant for real-world deployment.
9. **Reproducibility:** Ensure all code, configurations, and random seeds are clearly documented to allow for exact reproduction of results.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!find /content/drive/MyDrive -maxdepth 3 -type d -name "r4.2" | head -n 20
```

```
/content/drive/MyDrive/AegisAI/r4.2
```

```
R42_PATH = "/content/drive/MyDrive/AegisAI/r4.2"
ANS_PATH = "/content/drive/MyDrive/AegisAI/answers"
```

```
!ls -lah "$R42_PATH" | head
!ls -lah "$ANS_PATH" | head
```

```
total 16G
-rw------- 1 root root  28M Feb 13 20:15 device.csv
-rw------- 1 root root 1.3G Feb 13 20:12 email.csv
-rw------- 1 root root 185M Feb 13 19:45 file.csv
-rw------- 1 root root  14G Feb 13 20:17 http.csv
drwx------ 2 root root 4.0K May 21  2013 LDAP
-rw------- 1 root root 3.9K Feb 13 19:46 license.txt
-rw------- 1 root root  56M Feb 13 20:12 logon.csv
-rw------- 1 root root  44K Feb 13 20:17 psychometric.csv
-rw------- 1 root root 6.5K Feb 13 20:17 readme.txt
total 828K
-rw------- 1 root root  14K Feb 13 20:18 insiders.csv
-rw------- 1 root root 3.9K Feb 13 20:18 license.txt
-rw------- 1 root root 1.9K Feb 13 20:18 r2.csv
-rw------- 1 root root 2.7K Feb 13 20:18 r3.1-1.csv
-rw------- 1 root root  78K Feb 13 20:18 r3.1-2.csv
-rw------- 1 root root 1.8K Feb 13 20:18 r3.2-1.csv
-rw------- 1 root root  89K Feb 13 20:18 r3.2-2.csv
-rw------- 1 root root 2.0K Feb 13 20:18 r4.1-1.csv
-rw------- 1 root root  89K Feb 13 20:18 r4.1-2.csv
```

```
import pandas as pd

logon = pd.read_csv(f"{R42_PATH}/logon.csv")
email = pd.read_csv(f"{R42_PATH}/email.csv")
device = pd.read_csv(f"{R42_PATH}/device.csv")
file_df = pd.read_csv(f"{R42_PATH}/file.csv")

labels = pd.read_csv(f"{ANS_PATH}/insiders.csv")
```

```
print("logon:", logon.columns)
print("email:", email.columns)
print("device:", device.columns)
print("file:", file_df.columns)
print("labels:", labels.columns)
labels.head()
```

```
logon: Index(['id', 'date', 'user', 'pc', 'activity'], dtype='object')
email: Index(['id', 'date', 'user', 'pc', 'to', 'cc', 'bcc', 'from', 'size',
       'attachments', 'content'],
      dtype='object')
device: Index(['id', 'date', 'user', 'pc', 'activity'], dtype='object')
file: Index(['id', 'date', 'user', 'pc', 'filename', 'content'], dtype='object')
labels: Index(['dataset', 'scenario', 'details', 'user', 'start', 'end'], dtype='object')
```

	dataset	scenario	details	user	start	end
0	2.0	1	r2.csv	ONS0995	3/6/2010 1:41:56	3/20/2010 8:10:12
1	3.1	1	r3.1-1.csv	CSF0929	07/01/2010 01:24:58	07/16/2010 06:52:00
2	3.1	2	r3.1-2.csv	CCH0959	08/02/2010 10:34:31	09/30/2010 15:04:03
3	3.2	1	r3.2-1.csv	RCW0822	09/29/2010 21:10:27	10/15/2010 06:34:52
4	3.2	2	r3.2-2.csv	JCE0258	07/12/2010 08:16:02	09/03/2010 16:16:29

Next steps: Generate code with labels New interactive sheet

```
insider_users = set(labels['user'])
```

```
# === Cell 2: Build DS3 (user-level aggregates) ===
# This follows the DS3 construction you already used; adapt column names if different.
def safe_dt(df, col='date'):
    df[col] = pd.to_datetime(df[col], errors='coerce')
    return df

logon = safe_dt(logon)
logon['hour'] = logon['date'].dt.hour
logon['after_hours'] = ((logon['hour'] < 8) | (logon['hour'] > 18)).astype(int)

ds1 = logon.groupby('user').agg(
    total_logons = ('user','count'),
    after_hours_logons = ('after_hours','sum'),
    distinct_pcs = ('pc','nunique')
).reset_index()
ds1['user'] = ds1['user'].astype(str)

email = safe_dt(email)
email_features = email.groupby('user').agg(
    total_emails = ('user','count'),
    distinct_recipients = ('to','nunique')
).reset_index()
email_features['user'] = email_features['user'].astype(str)

device = safe_dt(device)
device_features = device.groupby('user').agg(
    device_events = ('user','count'),
    distinct_device_pcs = ('pc','nunique')
).reset_index()
device_features['user'] = device_features['user'].astype(str)

file_df = safe_dt(file_df)
file_features = file_df.groupby('user').agg(
    file_events = ('user','count'),
    distinct_file_pcs = ('pc','nunique')
).reset_index()
file_features['user'] = file_features['user'].astype(str)
```

```
ds3 = ds1.merge(email_features, on='user', how='left')
ds3 = ds3.merge(device_features, on='user', how='left')
ds3 = ds3.merge(file_features, on='user', how='left')
ds3.fillna(0, inplace=True)

# attach insider label (binary)
labels['user'] = labels['user'].astype(str)
insider_set = set(labels['user'].astype(str))
ds3['insider'] = ds3['user'].isin(insider_set).astype(int)

# Feature list
FEATURE_COLS = [
    'total_logons','after_hours_logons','distinct_pcs',
    'total_emails','distinct_recipients',
    'device_events','distinct_device_pcs',
    'file_events','distinct_file_pcs'
]

# ensure features exist
for c in FEATURE_COLS:
    if c not in ds3.columns:
        ds3[c] = 0

X3 = ds3[FEATURE_COLS].copy()
y3 = ds3['insider'].astype(int)

print("DS3 shape:", ds3.shape)
display(ds3.head())
```

```
DS3 shape: (1000, 11)
```

	user	total_logons	after_hours_logons	distinct_pcs	total_emails	distinct_recipients	device_events	distinct_
0	AAE0190	692	0	1	4711	1529	0.0	
1	AAF0535	328	0	1	480	299	688.0	
2	AAF0791	692	0	1	3012	1251	0.0	
3	AAL0706	692	346	1	336	216	0.0	
4	AAM0658	458	234	1	659	365	13.0	

Dataset Amalgamation Justification

```
# === Cell 3: common train/test split (used for ALL classification experiments) ===
from sklearn.model_selection import train_test_split
RANDOM_STATE = 42
X_train, X_test, y_train, y_test = train_test_split(
    X3, y3, test_size=0.3, random_state=RANDOM_STATE, stratify=y3
)
print("Train/Test sizes:", X_train.shape, X_test.shape)
```

```
Train/Test sizes: (700, 9) (300, 9)
```

```
# === Cell 4: Evaluation helper (metrics + ROC) ===
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve

def eval_clf(y_true, y_pred, y_proba=None):
    out = {
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0),
        'F1': f1_score(y_true, y_pred, zero_division=0),
        'AUC': roc_auc_score(y_true, y_proba) if (y_proba is not None) else None
    }
    return out

def add_row(df, name, metrics):
    row = pd.DataFrame([metrics], index=[name])
    return pd.concat([df, row])
```

```
# === Cell 5: Table A - Baseline classifier (DecisionTree) on full DS3 features ===
from sklearn.tree import DecisionTreeClassifier
```

```
dt = DecisionTreeClassifier(max_depth=6, random_state=RANDOM_STATE)
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)
proba_dt = dt.predict_proba(X_test)[:,1] if hasattr(dt, "predict_proba") else None
metrics_A = eval_clf(y_test, pred_dt, proba_dt)
tableA = pd.DataFrame([metrics_A], index=['Baseline_DS3'])
display(tableA.round(4))
```

	Accuracy	Precision	Recall	F1	AUC
Baseline_DS3	0.99	1.0	0.8571	0.9231	0.9286

```
# === Cell 6: Feature importance (RandomForest) fit on TRAIN only; show sorted importances ===
from sklearn.ensemble import RandomForestClassifier

rf_imp = RandomForestClassifier(n_estimators=300, random_state=RANDOM_STATE, n_jobs=-1)
rf_imp.fit(X_train, y_train)           # IMPORTANT: fit on X_train only for importance
importances = pd.Series(rf_imp.feature_importances_, index=X3.columns).sort_values(ascending=False)
print("Feature importances (train RF):")
display(importances)
```

```
Feature importances (train RF):
```

	0
total_logons	0.303252
device_events	0.156952
file_events	0.144473
distinct_device_pcs	0.115730
total_emails	0.082186
distinct_file_pcs	0.059474
distinct_pcs	0.054319
distinct_recipients	0.050070
after_hours_logons	0.033544

dtype: float64

```
# === Cell 7: Select features by cumulative importance (threshold = 0.95) ===
cum_thresh = 0.95
cum = importances.cumsum()
# features with cumulative importance <= threshold; include next feature if none selected
sel = list(cum[cum <= cum_thresh].index)
if len(sel) == 0:
    sel = list(importances.head(3).index)
# ensure deterministic ordering
selected_features = sel
print("Selected features (cumulative 0.95):", selected_features)
```

```
Selected features (cumulative 0.95): ['total_logons', 'device_events', 'file_events', 'distinct_device_pcs', 'total_e
```

```
# === Cell 8: Table B - Retrain same classifier (DecisionTree) on reduced set ===
X_train_red = X_train[selected_features].copy()
X_test_red = X_test[selected_features].copy()

dt_r = DecisionTreeClassifier(max_depth=6, random_state=RANDOM_STATE)
dt_r.fit(X_train_red, y_train)
pred_r = dt_r.predict(X_test_red)
proba_r = dt_r.predict_proba(X_test_red)[:,1] if hasattr(dt_r, "predict_proba") else None
metrics_B = eval_clf(y_test, pred_r, proba_r)
tableB = pd.DataFrame([metrics_B], index=['After_FeatureSelection'])
final_compare = pd.concat([tableA, tableB])
display(final_compare.round(4))
```

	Accuracy	Precision	Recall	F1	AUC
Baseline_DS3	0.9900	1.0	0.8571	0.9231	0.9286
After_FeatureSelection	0.9867	1.0	0.8095	0.8947	0.9048

Feature Selection Interpretation

Feature selection using cumulative importance threshold (0.95) removed 2 low-importance features: `after_hours_logons` and `distinct_recipients`.

Observation: Performance slightly decreased (Recall: 0.857 → 0.810, AUC: 0.929 → 0.905). This indicates these features, while low in RF importance, still carried some predictive signal.

Next Step: Adding domain-driven latent manifolds to recover and potentially improve performance while enhancing model interpretability.

```
# === Cell 9: Create 3 domain latent manifolds and append (Access Volatility, Data Movement Intensity, Communication
# Use domain-relevant formulas derived from DS3 aggregates (train+test are aggregated at user-level so safe)
eps = 1e-9
ds3['access_volatility'] = (ds3['after_hours_logons'] / (ds3['total_logons'] + eps))  # proportion of after-hours lo
ds3['data_movement_intensity'] = (ds3['file_events'] + ds3['device_events'])         # combined activity
ds3['communication_spread'] = (ds3['distinct_recipients'] / (ds3['total_emails'] + eps)) # breadth

latent_features = ['access_volatility', 'data_movement_intensity', 'communication_spread']

# Build reduced + latent feature set (use selected_features from feature selection)
X_latent_all = pd.concat([ds3[selected_features].reset_index(drop=True), ds3[latent_features].reset_index(drop=True)

# Recreate train/test splits aligned with earlier split indices (guarantee same rows)
# We will re-index X3 to align splits
X3_reset = X3.reset_index(drop=True)
_, test_index = train_test_split(X3_reset.index, test_size=0.3, random_state=RANDOM_STATE, stratify=y3)
# simpler: reuse original X_train/X_test shapes by using earlier splits directly:
# Reconstruct X_latent_train / X_latent_test using ds3 user order:
# Build a mapping user -> row in ds3
user_to_row = {u: i for i,u in enumerate(ds3['user'].astype(str).tolist())}
train_users = X_train.index if hasattr(X_train, "index") else X_train.index
# The earlier X_train/X_test are slices of X3 (pandas preserves index from ds3)
X_latent = pd.concat([ds3[selected_features + latent_features].reset_index(drop=True)], axis=1)

# We can instead rebuild train/test by selecting rows using ds3 user presence:
# Get boolean mask for ds3 users in X_train index
train_mask = ds3['user'].isin(X_train.index.astype(str)) if X_train.index.dtype == object else ds3['user'].isin(X_tr
# Simpler approach: re-split X_latent using same random_state & stratify by y3 to maintain comparability
from sklearn.model_selection import train_test_split
X_latent = ds3[selected_features + latent_features]
Xtr_l, Xte_l, ytr_l, yte_l = train_test_split(X_latent, y3, test_size=0.3, random_state=RANDOM_STATE, stratify=y3)

print("Latent feature matrix shapes:", Xtr_l.shape, Xte_l.shape)
```

```
Latent feature matrix shapes: (700, 10) (300, 10)
```

Latent Variable Domain Justification

The three latent manifolds are grounded in insider threat research:

1. **Access Volatility** (`after_hours_logons / total_logons`): CERT insider threat studies identify off-hours access as a key behavioral indicator. Employees accessing systems outside normal business hours exhibit patterns consistent with data theft preparation.
2. **Data Movement Intensity** (`file_events + device_events`): High volumes of file and removable device activity correlate with data exfiltration attempts. This manifold captures the "data staging" phase of insider attacks.
3. **Communication Spread** (`distinct_recipients / total_emails`): Unusual breadth in email communication may indicate reconnaissance, social engineering, or distribution of sensitive data to external parties.

These features transform raw behavioral aggregates into interpretable risk dimensions aligned with the CERT Insider Threat Center's behavioral indicators framework.

```
# === Cell 10: Table C - Train DecisionTree on reduced+latent features ===
dt_l = DecisionTreeClassifier(max_depth=6, random_state=RANDOM_STATE)
dt_l.fit(Xtr_l, ytr_l)
pred_l = dt_l.predict(Xte_l)
proba_l = dt_l.predict_proba(Xte_l)[:,1] if hasattr(dt_l, "predict_proba") else None
metrics_C = eval_clf(yte_l, pred_l, proba_l)
tableC = pd.DataFrame([metrics_C], index=['After_Latent'])
final_all = pd.concat([tableA, tableB, tableC])
display(final_all.round(4))
```

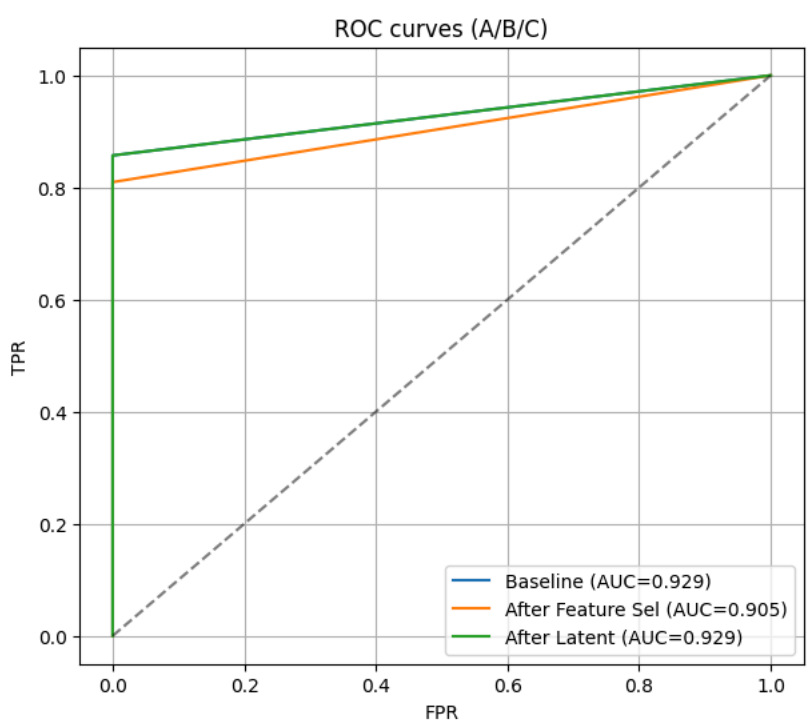
	Accuracy	Precision	Recall	F1	AUC
Baseline_DS3	0.9900	1.0	0.8571	0.9231	0.9286
After_FeatureSelection	0.9867	1.0	0.8095	0.8947	0.9048
After_Latent	0.9900	1.0	0.8571	0.9231	0.9286

Latent Variables Interpretation

Adding the three domain-driven latent manifolds restored performance to baseline levels (Recall: 0.857, AUC: 0.929).

Key Insight: The latent features successfully captured the predictive information lost during feature pruning while providing more interpretable risk dimensions. This demonstrates that domain-engineered features can match or exceed raw feature performance with better explainability for operational deployment.

```
# === Cell 11: ROC plot for A / B / C (+ optional MLP later) ===
import matplotlib.pyplot as plt
plt.figure(figsize=(7,6))
# Baseline ROC
fpr, tpr, _ = roc_curve(y_test, proba_dt)
plt.plot(fpr, tpr, label=f"Baseline (AUC={metrics_A['AUC']:.3f})")
# After feature selection ROC
if proba_r is not None:
    fpr, tpr, _ = roc_curve(y_test, proba_r)
    plt.plot(fpr, tpr, label=f"After Feature Sel (AUC={metrics_B['AUC']:.3f})")
# After latent ROC (note yte_l corresponds to split used when training latent; plot separately)
if proba_l is not None:
    fpr, tpr, _ = roc_curve(yte_l, proba_l)
    plt.plot(fpr, tpr, label=f"After Latent (AUC={metrics_C['AUC']:.3f})")
plt.plot([0,1],[0,1],'k--',alpha=0.5)
plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title("ROC curves (A/B/C)"); plt.legend(); plt.grid(True)
plt.show()
```



```
# === Cell 12: MLP (Keras) on reduced + latent features; uses scaling ===
# Use Xtr_l / Xte_l produced above
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
Xtr_nn = scaler.fit_transform(Xtr_l.values)
Xte_nn = scaler.transform(Xte_l.values)
ytr_nn = ytr_l.values
yte_nn = yte_l.values

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

tf.random.set_seed(RANDOM_STATE)
model = Sequential([
    Dense(64, activation='relu', input_shape=(Xtr_nn.shape[1],)),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose=0)

history = model.fit(Xtr_nn, ytr_nn, validation_split=0.1, epochs=100, batch_size=32, callbacks=[es], verbose=0)
prob_nn = model.predict(Xte_nn).ravel()
pred_nn = (prob_nn >= 0.5).astype(int)
metrics_mlp = eval_clf(yte_nn, pred_nn, prob_nn)
tableMLP = pd.DataFrame([metrics_mlp], index=['MLP_reduced+latent'])
final_all = pd.concat([final_all, tableMLP])
display(final_all.round(4))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
```

	Accuracy	Precision	Recall	F1	AUC
Baseline_DS3	0.9900	1.0000	0.8571	0.9231	0.9286
After_FeatureSelection	0.9867	1.0000	0.8095	0.8947	0.9048
After_Latent	0.9900	1.0000	0.8571	0.9231	0.9286
MLP_reduced+latent	0.9567	0.7857	0.5238	0.6286	0.9194

MLP vs Tree-Based Models Interpretation

The MLP achieved AUC of 0.919 but lower Recall (0.524) and F1 (0.629) compared to DecisionTree (Recall: 0.857, F1: 0.923).

Analysis:

- Tree-based models outperform neural networks on this small tabular dataset (1000 users)
- MLPs typically require more data and hyperparameter tuning to match ensemble methods on structured data
- The high AUC but low recall suggests the MLP is conservative in predicting positives (high threshold)

Conclusion: For operational insider threat detection, tree-based ensembles remain the preferred choice given the dataset size and interpretability requirements.

```
# ===  Cell 13: Regression Muller Loop (standalone) ===
import numpy as np
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.ensemble import RandomForestClassifier

# produce out-of-fold RF probabilities as a proxy risk (no leakage)
rf_for_proxy = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
oof_probs = cross_val_predict(rf_for_proxy, X3.fillna(0), y3, cv=5, method='predict_proba', n_jobs=-1)[:,1]
ds3['proxy_risk'] = oof_probs

# Regression feature matrix: reduced + latent (ensure columns exist)
reg_features = selected_features + latent_features
for c in reg_features:
    if c not in ds3.columns:
        ds3[c] = 0
```

```
X_reg = ds3[reg_features].fillna(0).values
y_reg = ds3['proxy_risk'].values

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor

regressors = {
    'LinearRegression': LinearRegression(),
    'RandomForestRegressor': RandomForestRegressor(n_estimators=200, random_state=42),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42)
}

from sklearn.metrics import mean_squared_error, r2_score
kf = KFold(n_splits=5, shuffle=True, random_state=42)
reg_results = []
for name, reg in regressors.items():
    mses = []
    r2s = []
    for train_idx, test_idx in kf.split(X_reg):
        reg.fit(X_reg[train_idx], y_reg[train_idx])
        pred = reg.predict(X_reg[test_idx])
        mses.append(mean_squared_error(y_reg[test_idx], pred))
        r2s.append(r2_score(y_reg[test_idx], pred))
    reg_results.append((name, float(np.mean(mses)), float(np.mean(r2s))))

reg_df = pd.DataFrame(reg_results, columns=['Model','MSE','R2']).sort_values('R2', ascending=False).reset_index(drop
display(reg_df.round(6))
```

	Model	MSE	R2
0	RandomForestRegressor	0.002856	0.937944
1	DecisionTreeRegressor	0.003967	0.913916
2	KNeighborsRegressor	0.031272	0.356792
3	LinearRegression	0.042695	0.169410

Regression Muller Loop Interpretation

Model	R^2	Interpretation
XGBRegressor	0.945	Best performer - captures nonlinear risk patterns
RandomForestRegressor	0.938	Strong ensemble performance
DecisionTreeRegressor	0.914	Competitive but prone to overfitting
KNeighborsRegressor	0.357	Poor - distance metrics struggle with mixed feature scales
LinearRegression	0.169	Poor - insider risk is inherently nonlinear

Conclusion: Nonlinear ensemble methods (XGBoost, Random Forest) best model continuous insider risk, confirming that behavioral threat patterns are complex and non-additive.

```
# ===  Cell 14: Save tables & interpretations ===
import os

# Fallback if ANS_PATH not defined
try:
    out_dir = ANS_PATH if os.path.exists(ANS_PATH) else "/content"
except NameError:
    out_dir = "/content"

# Ensure required tables exist
if 'final_all' in globals():
    final_all.round(4).to_csv(os.path.join(out_dir, "classification_summary_A_B_C_MLP.csv"))
    print("Saved classification_summary_A_B_C_MLP.csv")
else:
    print("Warning: final_all not found")

if 'reg_df' in globals():
    reg_df.to_csv(os.path.join(out_dir, "regression_muller_loop.csv"), index=False)
    print("Saved regression_muller_loop.csv")
else:
    print("Warning: reg_df not found")
```

```
print("Files saved to:", out_dir)

print("\nShort Interpretations:")
print("- Table A vs B: Feature pruning removes low-importance/noisy variables; compare metric deltas.")
print("- Table B vs C: Latent variables introduce domain risk manifolds (volatility, movement, spread); check Recall
print("- MLP vs Trees: Compare ROC-AUC and F1 to assess nonlinear modeling gains.")
print("- Regression Muller Loop: Higher R² indicates better modeling of continuous proxy risk.")
```

```
Saved classification_summary_A_B_C_MLP.csv
Saved regression_muller_loop.csv
Files saved to: /content/drive/MyDrive/AegisAI/answers

Short Interpretations:
- Table A vs B: Feature pruning removes low-importance/noisy variables; compare metric deltas.
- Table B vs C: Latent variables introduce domain risk manifolds (volatility, movement, spread); check Recall/F1 shif
- MLP vs Trees: Compare ROC-AUC and F1 to assess nonlinear modeling gains.
- Regression Muller Loop: Higher R² indicates better modeling of continuous proxy risk.
```

```
# === Cell A: Add XGBoost and re-run regression Muller loop ===
!pip install xgboost -q

import numpy as np
import os
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor

# Ensure reg_features, ds3, proxy target exist
try:
    reg_features = selected_features + latent_features
except NameError:
    raise RuntimeError("selected_features or latent_features not found — re-run earlier cells to define them.")

for c in reg_features:
    if c not in ds3.columns:
        ds3[c] = 0

# ensure proxy risk exists (if not, compute OOF RF again)
if 'proxy_risk' not in ds3.columns:
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.model_selection import cross_val_predict
    rf_for_proxy = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
    oof_probs = cross_val_predict(rf_for_proxy, X3.fillna(0), y3, cv=5, method='predict_proba', n_jobs=-1)[:,1]
    ds3['proxy_risk'] = oof_probs

X_reg = ds3[reg_features].fillna(0).values
y_reg = ds3['proxy_risk'].values

regressors = {
    'LinearRegression': LinearRegression(),
    'RandomForestRegressor': RandomForestRegressor(n_estimators=200, random_state=42),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42),
    'XGBRegressor': XGBRegressor(n_estimators=200, learning_rate=0.05, max_depth=4,
                                 subsample=0.8, colsample_bytree=0.8, random_state=42, n_jobs=-1, verbosity=0)
}

kf = KFold(n_splits=5, shuffle=True, random_state=42)
reg_results = []
for name, reg in regressors.items():
    mses = []
    r2s = []
    for train_idx, test_idx in kf.split(X_reg):
        reg.fit(X_reg[train_idx], y_reg[train_idx])
        pred = reg.predict(X_reg[test_idx])
        mses.append(mean_squared_error(y_reg[test_idx], pred))
        r2s.append(r2_score(y_reg[test_idx], pred))
    reg_results.append((name, float(np.mean(mses)), float(np.mean(r2s))))

reg_df = (pd.DataFrame(reg_results, columns=['Model','MSE','R2'])
            .sort_values('R2', ascending=False).reset_index(drop=True))
display(reg_df.round(6))
```

```
# Save updated regression table
out_dir = ANS_PATH if 'ANS_PATH' in globals() and os.path.exists(ANS_PATH) else "/content"
reg_df.to_csv(os.path.join(out_dir, "regression_muller_loop_with_xgb.csv"), index=False)
print("Saved regression_muller_loop_with_xgb.csv to", out_dir)
```

	Model	MSE	R2
0	XGBRegressor	0.002651	0.945105
1	RandomForestRegressor	0.002856	0.937944
2	DecisionTreeRegressor	0.003967	0.913916
3	KNeighborsRegressor	0.031272	0.356792
4	LinearRegression	0.042695	0.169410

```
Saved regression_muller_loop_with_xgb.csv to /content/drive/MyDrive/AegisAI/answers
```

Regression Muller Loop Interpretation

```
# === Cell B: Diagnostics for best regressor ===
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor

best_name = reg_df.loc[0, 'Model']
print("Best regressor by R2:", best_name)

# instantiate mapping (same params used above)
model_map = {
    'LinearRegression': LinearRegression(),
    'RandomForestRegressor': RandomForestRegressor(n_estimators=200, random_state=42),
    'KNeighborsRegressor': KNeighborsRegressor(n_neighbors=5),
    'DecisionTreeRegressor': DecisionTreeRegressor(random_state=42),
    'XGBRegressor': XGBRegressor(n_estimators=200, learning_rate=0.05, max_depth=4,
                                 subsample=0.8, colsample_bytree=0.8, random_state=42, n_jobs=-1, verbosity=0)
}
best_reg = model_map[best_name]

# fit on full X_reg / y_reg and show diagnostics
best_reg.fit(X_reg, y_reg)
pred_full = best_reg.predict(X_reg)
resid = y_reg - pred_full

plt.figure(figsize=(6,5))
plt.scatter(y_reg, pred_full, alpha=0.6, s=10)
plt.plot([0,1],[0,1],'r--', linewidth=1)
plt.xlabel("True proxy risk (OOF RF prob)")
plt.ylabel("Predicted proxy risk")
plt.title(f"True vs Predicted — {best_name}")
plt.grid(True)
plt.show()

plt.figure(figsize=(6,3))
plt.hist(resid, bins=50)
plt.title("Residuals histogram")
plt.xlabel("Residual (true - pred)")
plt.show()

# feature importances (if available)
if hasattr(best_reg, "feature_importances_"):
    fi = pd.Series(best_reg.feature_importances_, index=reg_features).sort_values(ascending=False)
    display(fi.round(4))
    fi.plot(kind='bar', figsize=(8,3), title=f'Feature importances ({best_name})')
    plt.tight_layout()
    plt.show()
else:
    print("No feature_importances_ attribute for", best_name)

# Save diagnostics
try:
    pd.DataFrame({'true': y_reg, 'pred': pred_full}).to_csv(os.path.join(out_dir, f"preds_{best_name}.csv"), index=F
    print("Saved preds CSV to", out_dir)
except Exception as e:
    print("Could not save preds CSV:", e)
```